

Discorso di laurea · 15 minuti

Leggi con calma, come se lo raccontassi. La barra azzurra con il triangolo indica quando cambiare slide; a destra vedi la slide che deve comparire.

► INIZIO · SLIDE 1

Copertina



Buongiorno. Ringrazio il presidente e tutta la commissione per la parola. Sono Claudio Dragotta e vi presento la mia tesi, che ho svolto con il professor Soda e l'ingegner Caprio.

► CAMBIA SLIDE · 2

La tesi in sintesi



Parto da un dato oggettivo. I computer quantistici lavorano sugli stati quantistici della materia, che sono fragilissimi: basta una minima perturbazione per alterarli. Per questo un computer quantistico non è mai preciso al cento per cento, e anche le macchine future dovranno convivere con gli errori.

Lo scopo della mia tesi è stato analizzare l'algoritmo di Shor proprio in queste condizioni, e capire quali strategie gli permettono di dare comunque buoni risultati: il machine learning sui risultati, la correzione degli errori e la riduzione delle porte. Tutte e tre possono aiutare, ma ognuna solo in condizioni precise.

► CAMBIA SLIDE · 3

Indice



Il percorso è questo: il contesto e gli obiettivi, i metodi, i risultati e infine le conclusioni.

► CAMBIA SLIDE · 4

Il problema: il rumore mette in crisi Shor



Partiamo dal perché. La sicurezza di RSA, che protegge buona parte delle nostre comunicazioni, si basa sul fatto che fattorizzare un numero molto grande è difficilissimo per un computer classico. Shor ha dimostrato che un computer quantistico ideale lo farebbe in tempo polinomiale. Ma sulle macchine reali il rumore altera proprio l'informazione da cui Shor ricava la soluzione.

► CAMBIA SLIDE · 5

Come funziona l'algoritmo di Shor



Vediamo come funziona, in modo rigoroso. Abbiamo N , il numero da fattorizzare, e un parametro a , la base, che non ha divisori in comune con N . La funzione a alla x modulo N è periodica, e il compito del computer quantistico è trovarne il periodo r . Nel mio esempio N è 15 e a è 7: nel grafico i valori si ripetono ogni quattro passi, quindi r è 4. Da r un calcolo classico ricava un fattore, 3, e quindi anche l'altro, 5.

Il circuito usa 8 qubit di controllo e 4 di lavoro: 12 qubit in tutto.

► CAMBIA SLIDE · 6

Obiettivi: tre livelli di intervento



La letteratura propone tre strade contro il rumore: ripulire i risultati dopo la misura, proteggere i qubit con la correzione degli errori, o semplificare il circuito. Raramente però vengono confrontate con un riferimento adeguato. Da qui i miei tre obiettivi, ognuno con il suo riferimento: capire se il machine learning migliora davvero la lettura dei risultati, confrontare codici di correzione degli errori e verificare se togliere porte migliora il risultato finale.

► CAMBIA SLIDE · 7

Dove interviene ciascuna strategia



Questo schema riassume dove ho lavorato. In giallo c'è l'algoritmo di Shor, con i suoi passi: scelta di a , esponenziazione modulare, QFT inversa e misura. In viola ci sono i miei interventi.

La QFT approssimata agisce sulla QFT inversa, per ridurre le porte. Il post-processing agisce dopo la misura, ed è lì che ho usato il machine learning. La correzione degli errori, invece, serve a proteggere i qubit durante il calcolo vero e proprio, cioè nell'esponenziazione e nella QFT. Preciso che questa parte l'ho studiata in esperimenti separati, non dentro il circuito di Shor.

► CAMBIA SLIDE · 8

Come si misura il successo di Shor



Prima di vedere i risultati, bisogna sapere come leggerli. Nel caso ideale Shor trova i fattori nel 75% dei casi.

Ma c'è un secondo valore da tenere presente, il 24,61%: è la percentuale che otterremmo anche con risultati completamente casuali, perché la verifica finale accetta 63 esiti su 256, circa un quarto. Quindi un risultato vicino a questo valore non significa che Shor stia funzionando: tutto va letto tra questi due estremi.

► CAMBIA SLIDE · 9

Istanze, validazione e strumenti



Ho lavorato su due casi. Il principale è N uguale a 15, con 12 qubit, su cui ho fatto tutte le prove con rumore.

Il secondo è N uguale a 21, che richiede 20 qubit: 8 di controllo, 5 di lavoro, perché 21 si scrive con 5 bit, e 7 ausiliari per i calcoli modulari. Su questo ho verificato il circuito e fatto alcune prove sulla QFT approssimata. Tutto è simulato al calcolatore, con strumenti open source.

► CAMBIA SLIDE · 10

Metodo 1: post-processing e ablazione ML



Il primo metodo agisce dopo la misura. Ogni esecuzione di Shor dà un istogramma di risultati, e bisogna decidere quali verificare.

TOP-1 verifica solo il più frequente. TOP-4 verifica, uno alla volta, i quattro più frequenti, finché trova i fattori. Il Metodo 2 applica TOP-4 solo se un classificatore di machine learning lo approva. La verifica finale è sempre la stessa, un calcolo classico.

In letteratura il machine learning è stato proposto proprio per filtrare questi risultati; il rischio è dargli un merito che viene invece dal provare più candidati. Per questo il confronto decisivo è tra il Metodo 2 e TOP-4.

► CAMBIA SLIDE · 11

Metodo 2: correzione quantistica degli errori



Il secondo metodo è la correzione quantistica degli errori: più qubit fisici proteggono un solo qubit logico, i controlli segnalano dove c'è un errore e un decoder decide come correggerlo.

Ho usato due codici. Il surface code è il riferimento, robusto ma costoso: per 12 qubit logici servono circa 2000 qubit di dato. Su questo ho confrontato i decoder: quello classico, MWPM, una rete neurale, un ibrido e BP+OSD.

Il Gross code, proposto da IBM, promette una protezione simile con circa 14 volte meno qubit. Confrontarli dice quanto si può risparmiare. Sul Gross code ho simulato gli errori come sequenze di 144 bit, non lo stato di 144 qubit.

► CAMBIA SLIDE · 12

Metodo 3: QFT approssimata (AQFT)



Il terzo metodo nasce da un'osservazione semplice: gli errori vengono dalle porte, quindi meno porte significa meno errori.

Nella QFT ci sono rotazioni sempre più piccole, che spostano la fase quasi di nulla ma costano porte. Nella versione approssimata le tolgo; per N uguale a 15 bastano quelle tra qubit vicini. Funziona perché Shor non ha bisogno della

fase esatta: basta un valore abbastanza vicino perché il calcolo classico ritrovi il periodo.

L'ho provato su N uguale a 15, su N uguale a 21 e su test di stima di fase, con il rumore di un computer IBM reale.

► CAMBIA SLIDE · 13

Quanto il rumore degrada Shor



Passiamo ai risultati. Per simulare il rumore, dopo ogni porta inserisco con una certa probabilità un operatore di Pauli casuale, cioè un'inversione del bit, della fase o di entrambi.

Nel grafico, andando da sinistra a destra cresce l'errore per porta; in alto c'è il 75% ideale, in basso la linea tratteggiata del caso. Senza rumore il successo è del 74,89%, come previsto. Basta l'1% di errore per porta, il punto cerchiato, per scendere al 56%, e oltre il 10% la curva si appiattisce sulla linea del caso: lì Shor non funziona più.

► CAMBIA SLIDE · 14

Risultato 1: il beneficio viene da TOP-4



Il primo risultato riguarda il machine learning. Nel grafico, barre più basse significano meno tentativi per trovare i fattori. Guardate il riquadro arancione: TOP-4 trova i fattori sempre al primo tentativo, in tutte le 30 repliche.

Il classificatore è molto accurato, eppure il Metodo 2 non abbassa le barre, anzi fa peggio di TOP-4, perché a volte scarta proprio i risultati buoni.

Quindi il vantaggio viene dal provare più candidati, non dal machine learning.

► CAMBIA SLIDE · 15

Risultato 2: la QEC aiuta sotto soglia



Il secondo risultato riguarda la correzione degli errori. Ogni colore è un codice più grande. Nella zona verde, sotto soglia, le curve dei codici grandi stanno più in basso: fanno meno errori. Nella zona rossa l'ordine si inverte e aggiungere qubit peggiora le cose.

La soglia è il punto in cui le curve si incrociano, nei cerchi arancioni: l'ho

stimata tra lo 0,8 e lo 0,9% di errore. Anche i codici più semplici, a ripetizione e di Steane, si comportano come prevede la teoria.

► CAMBIA SLIDE · 16

Risultato 2: decoder ibrido con crosstalk



Qui il machine learning trova invece un ruolo. Nel grafico, sopra la linea dell'1 il decoder sbaglia meno di quello classico. Da sola la rete non lo batte; ma quando c'è crosstalk, cioè errori correlati tra qubit vicini, il decoder ibrido, la linea arancione continua, resta sopra: nel punto cerchiato gli errori scendono del 16,8%.

Sui codici più grandi, le linee tratteggiate, il vantaggio quasi sparisce. Quindi il machine learning aiuta come correttore del decoder classico, non come sostituto.

► CAMBIA SLIDE · 17

Risultato 2: Gross code e numero di qubit



E il confronto tra i due codici. Per proteggere 12 qubit logici il Gross code, la barra arancione, usa 144 qubit, circa 14 volte meno del surface code. Nel modello semplificato che ho usato, con un errore fisico dallo 0,75% in su, protegge anche meglio. E anche qui il decoder è decisivo: scegliendo la versione giusta, gli errori scendono di circa cento volte.

► CAMBIA SLIDE · 18

Risultato 3: AQFT su $N = 15$



Il terzo risultato è sulla QFT approssimata. Nel grafico, a destra c'è la QFT piena, a sinistra si tolgono sempre più rotazioni. Il grado scelto, quello cerchiato, sta ben sopra la QFT piena: il successo passa dal 48 al 63%, con quasi il 23% di porte in meno.

Va detto però che 15 è un caso favorevole. Su N uguale a 21 le porte si dimezzano, ma il vantaggio non è dimostrato: l'approssimazione va scelta caso per caso.

Conclusioni e contributo personale



Per concludere: una tecnica vale quanto l'informazione che riesce a sfruttare, e va sempre confrontata con un riferimento adeguato.

Il post-processing aiuta grazie ai candidati in più, non grazie al machine learning; la correzione degli errori funziona solo sotto soglia; la QFT approssimata aiuta, ma non sempre.

Il mio contributo è stato costruire e verificare i circuiti per N uguale a 15 e 21, progettare confronti corretti e rendere tutto riproducibile.

Limiti e sviluppi futuri



I limiti sono chiari: ho lavorato in simulazione, su pochi casi, e senza un'esecuzione completamente protetta dagli errori. Il passo successivo è ridurre ancora il costo dei circuiti e poi unirli alla correzione degli errori. Prima di chiudere, vi mostro in due minuti una demo che ho sviluppato, che riassume visivamente tutto il percorso.

Demo: Shor passo per passo, con e senza rumore



[Il video parte da solo e dura 2 minuti. Parla sopra il video, seguendo le scritte in basso; i tempi tra parentesi sono quelli del video.]

[0:00] La demo segue Shor passo per passo, prima senza errori e poi con il rumore.

[0:06] Il caso è quello della tesi: N uguale a 15, base 7. Ogni riga è un qubit, e a destra c'è la sua sfera di Bloch, che mostra lo stato del qubit con una freccia.

[0:17] Le porte di Hadamard mettono i qubit in sovrapposizione. Poi le moltiplicazioni per 7 modulo 15 scrivono il periodo nelle fasi; le frecce si accorciano, ma qui non è un errore: è entanglement.

[0:30] La QFT inversa trasforma il periodo in quattro picchi, e alla misura il calcolo classico ricava 15 uguale 3 per 5. Riesce tre volte su quattro, altrimenti

si ripete.

[0:48] Ora lo stesso circuito con il rumore. Guardate le sfere: le frecce che prima restavano intere si accorciano, lo stato perde coerenza. La demo lo misura: la lunghezza media scende da 1 a 0,81.

[1:08] Qui si vede dove entra il rumore: ogni porta ha il suo errore, le porte a due qubit sono le più fragili e la misura può sbagliare il bit.

[1:15] Facendo girare il circuito 512 volte con e senza rumore, nella demo i fattori si trovano nel 74% dei casi senza rumore e nel 56% con il rumore; tirando a caso si otterrebbe il 25%. Le misure si spargono su tutti gli esiti e i picchi si abbassano.

[1:33] Questa è informazione persa, che dopo non si recupera. La correzione degli errori aiuta solo sotto soglia, e Shor resta sensibile all'errore che rimane.

[1:53] Il messaggio della demo è lo stesso della tesi: il rumore va affrontato durante il calcolo, perché dopo la misura l'informazione distrutta non si ricostruisce.

► CAMBIA SLIDE · 22

Grazie per l'attenzione



Vi ringrazio per l'attenzione. Sono a disposizione per le vostre domande.

Domande probabili della commissione

Come hai simulato 144 qubit?

Non ho simulato 144 qubit: lo stato quantistico di 144 qubit richiederebbe circa 10^{44} byte, ed è impossibile. Ho testato l'algoritmo di correzione del Gross code. Ogni prova è una sequenza di 144 bit che indica su quali qubit di dato c'è un errore. Da questa calcolo la sindrome, cioè le misure di controllo, la passo al decoder BP+OSD e verifico se l'errore viene corretto. È un calcolo binario che si fa su un normale PC.

È confrontabile con i risultati di IBM?

No. Conto i soli qubit di dato, la sindrome è perfetta e il modello è semplificato, il cosiddetto code-capacity. È un confronto esplorativo tra codici.

Quanti qubit usa $N = 21$?

20: 8 di controllo come per $N = 15$, 5 di lavoro perché 21 si scrive con 5 bit, e 7 ausiliari per l'aritmetica modulare di Beauregard (6 di accumulatore e 1 ancilla).

Perché la correzione degli errori non è dentro Shor?

Simulare Shor completo con la correzione degli errori è fuori dalla portata di un calcolatore classico. L'ho studiata in esperimenti di memoria, dove la soglia ha un significato preciso. Unirle è il passo futuro.

E su $N = 21$, l'AQFT?

Le porte a due qubit passano da 49 661 a 23 178, ma il successo ideale scende dal 46,7 al 40,1%. Nelle prove con rumore la differenza non è significativa: il beneficio non è dimostrato.

Perché il classificatore non aiuta se è accurato?

Perché predice solo se il risultato più frequente è giusto, mentre TOP-4 prova comunque fino a quattro candidati. Quando il filtro scarta un istogramma, perde casi in cui il secondo o il terzo candidato avrebbe funzionato.

Che cos'è il rumore di Pauli?

È il modello standard degli errori: dopo una porta, con una certa probabilità, si applica per sbaglio un'operazione X (inversione del bit), Z (inversione della fase) o Y (entrambe). La probabilità per porta è p_g .

Che cosa significa la soglia?

È il livello di errore fisico sotto il quale la correzione conviene: sotto soglia, codici più grandi riducono l'errore logico; sopra soglia lo aumentano. Per il surface code l'ho stimata tra lo 0,8 e lo 0,9%.

Il plateau al 24,6% vuol dire che resta informazione?

No. È il livello che la verifica classica raggiunge anche con esiti casuali, perché accetta 63 esiti su 256.

Perché proprio $N = 15$?

È il caso standard negli studi su Shor e permette una campagna con rumore completa. È anche favorevole, perché il periodo 4 dà fasi esatte su due bit: per questo i risultati vanno letti come limitati a questa istanza.

Su quanti dati hai addestrato e testato il classificatore?

2000 istogrammi per ciascuno dei due scenari di rumore, divisi 60/20/20 tra addestramento, selezione e test (400 istogrammi di test indipendenti). Il confronto tra le strategie usa 30 repliche appaiate, con al massimo 50 iterazioni, e tutte ricevono gli stessi istogrammi.

Che cosa significa la sigla $[[144,12,12]]$?

È la notazione dei codici quantistici $[[n, k, d]]$: $n = 144$ qubit di dato, $k = 12$ qubit logici protetti, $d = 12$ distanza del codice.

Quali sono i riferimenti principali?

Per il machine learning sugli esiti di Shor, Yang e Markidis (2026); per i decoder appresi, Bausch et al. (Google DeepMind, Nature 2024); per BP+OSD, Roffe et al. (2020); per il Gross code, Bravyi et al. (IBM, Nature 2024); per la QFT approssimata, Barenco et al. (1996).

E i codici più semplici?

Il codice a ripetizione segue la formula teorica $p_L = 3p^2 - 2p^3$; il codice di Steane mostra la soppressione quadratica attesa, con pendenza 1,94 e pseudo-soglia circa 0,08.

Perché per $N = 21$ non sono 13 qubit ($2n + 3$)?

La formula $2n + 3$ di Beauregard vale con un solo qubit di controllo riciclato con misure intermedie. Io ne uso 8, uno per ogni bit della fase: 8 di controllo + 5 di lavoro + 6 di accumulatore + 1 ancilla = 20.

I numeri della demo sono risultati della tesi?

No. La demo usa seed 42 e il preset UC1, un modello di rumore uniforme e illustrativo, con 512 esecuzioni: serve a far vedere il fenomeno. I risultati della tesi sono quelli delle slide precedenti.